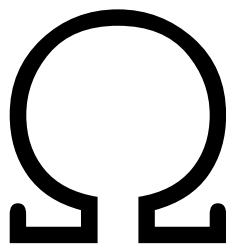


Society Protocol

Final Audit Report

May 8, 2026



Team Omega

`Teamomega.eth.limo`

Summary	3
Scope of the Audit	3
Methods Used	4
Resolution	4
Disclaimer	4
Severity definitions	4
Findings	5
General	5
G1. Contract dependencies are not pinned [low] [resolved]	5
G2. Test Coverage is not complete [low] [not resolved]	6
G3. Permission management is complex [info] [not resolved]	6
SocietyProtocolBadges.sol	7
SPB1. Profiles are not guaranteed to be “soul-bound” [high] [resolved]	7
SPB2. CreateProfile is vulnerable to reentrancy [medium] [resolved]	9
SPB3. Invitations do not implement EIP712 [medium][resolved]	10
SPB4. UpdateProfileURI allow for any badges with a supply of 1 to be updated by their owner [medium] [resolved]	10
SPB5. Override of balanceOf by hook may break expectations about token ownership [low] [resolved]	11
SPB6. Circular invite protection is only one-hop [low] [not resolved]	12
SPB7. HookUpdated event is not emitted when hook is cleared [low] [resolved]	12
SPB8. acceptInvite does not check if the inviter is an existing protocol user [info] [resolved]	12
SPB9. Permissions may break common expectations regarding ERC1155 [info] [resolved]	13
SPB10. The badgeCreated mapping can be safely removed [info] [resolved]	13
SPB11. Misleading variable names [info] [resolved]	14
SPB12. Editors, minters, transferers and burners can not be changed after badge creation [info] [resolved]	14
CommunityRegistry.sol	14
CR1. createCommunity is vulnerable to re-entry [low] [resolved]	14
CR2. Permissions as described in docstring do not correspond with actual permissions [info] [resolved]	15
CommunityWrapper.sol	16
CW1. Implementation of totalSupply limits possible Snapshot strategies that could be used [info] [resolved]	16
CW2. Approval does not throw an error [info] [resolved]	16
CommunityWrapperFactory.sol	16
CW1. Sanity check on implementation contract can be improved [low] [resolved]	16
SocietyVipManager.sol	17
SVM1. Staked Tokens can get locked forever [high] [resolved]	17

Summary

Society Protocol has asked Team Omega to audit the contracts that define the behavior of a Reward Pool

We found **2 high severity issues** - these are issues that can lead to a loss of funds or corruption of data, and are essential to fix. We classified **3** issues as “medium” - these are issues we believe you should definitely address. We did classify **7** issues as “low”, and an additional **9** issues were classified as “info” - we believe the code would improve if these issues were addressed as well.

All significant issues were resolved.

Severity	Number of issues	Number of resolved issues
High	2	2
Medium	3	3
Low	7	5
Info	9	8

Scope of the Audit

The audit concerns the Solidity Contracts in the following repository:

<https://github.com/SocietyProtocol/web3-app-contracts>

And specifically, the main branch which is currently at commit

`c9b663b0f00a55bc41f41ecd7260ce86f9534134`

At the moment of writing, the solidity files contain almost 800 nsloc of code (these “real lines of code” - not counting comments or empty lines or line wraps) in the following files:

```
contracts
├── Accounts
│   ├── CommunityRegistry.sol
│   ├── CommunityWrapper.sol
│   ├── CommunityWrapperFactory.sol
│   ├── ISocietyBadgeHook.sol
│   ├── SocietyProtocolBadges.sol
│   └── SocietyVipManager.sol
└── SPEC.sol
```

Methods Used

The contracts were compiled, deployed, and tested in a test environment.

Code Review

We manually inspected the source code to identify potential security flaws. We also inspected the context in which the contracts were developed..

Automatic analysis

We have used static analysis tools to detect common potential vulnerabilities, and had assistance from AI tools.

Resolution

After we delivered a preliminary report, most of the issues were addressed by the team in the following commit:

```
b23c7b4629a791326a284dd2a7fb5db561a4cdc3
```

We checked for each of the issues in our report if the issue was addressed (or not) and marked our observations below. We also audited the other changes that were made in this commit.

Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

Severity definitions

High	Vulnerabilities that can lead to loss of assets or data manipulations.
Medium	Vulnerabilities that are essential to fix, but that do not lead to assets loss or data manipulations

Low	Issues that do not represent direct exploit, such as poor implementations, deviations from best practice, high gas costs, etc
Info	Matters of opinion

Findings

General

G1. Contract dependencies are not pinned [low] [resolved]

In `package.json`, the contract dependencies specify a range of versions rather than a specific version:

```
"dependencies": {
  "@openzeppelin/contracts": "^5.0.0",
  "@openzeppelin/contracts-upgradeable": "^5.4.0"
}
```

This means that when a new (minor) version is released, a fresh installation of the package may use a more recent version. The risks here are of two types. First of all, the newer version may contain changes that are not compatible with the current version of the contracts, or introduce new vulnerabilities. The second problem is that it will be hard to verify that the bytecode of the contracts that are deployed on-chain corresponds with the code in the repository.

Recommendation: Pin the dependencies to a specific version, for example the version that is currently set in `package-lock.json`:

```
"dependencies": {
  "@openzeppelin/contracts": "5.4.0",
  "@openzeppelin/contracts-upgradeable": "5.4.0"
}
```

Severity: Low

Resolution: The issue was addressed as recommended

G2. Test Coverage is not complete [low] [not resolved]

Test coverage of the contracts is not complete.

all files contracts/Accounts/									
97.84% Statements 226/231 78.04% Branches 307/234 98.48% Functions 63/66 98.36% Lines 388/395									
File	Statements	Branches	Functions	Lines					
CommunityRegistry.sol	100%	35/35	76.92%	20/26	100%	13/13	100%	55/55	
CommunityWrapper.sol	100%	28/28	85%	17/20	100%	9/9	100%	33/33	
CommunityWrapperFactory.sol	100%	11/11	50%	6/12	100%	5/5	100%	14/14	
ISocietyBadgeHook.sol	100%	0/0	100%	0/0	100%	0/0	100%	0/0	
SocietyProtocolBadges.sol	96.43%	108/112	82.98%	78/94	96.15%	25/26	97.93%	142/145	
SocietyVipManager.sol	97.78%	44/45	74.19%	46/62	100%	13/13	96.55%	56/58	

It is important to have a complete test suite that covers all code paths, not only for confirming that the code works as expected, but also to “future-proof” the code, so that in future upgrades, the chances that something breaks unnoticed as minimal as possible

Resolution: Write tests that cover all code paths.

Severity: Low

Resolution: The issue was not resolved

G3. Permission management is complex [info] [not resolved]

The set of contracts defines a number of different roles that have the power to change essential parts of the system.

- The owner of `CommunityRegistry`, `CommunityWrapperFactory` and `SocietyVipManager` can upgrade the contracts.
- The owner of the `CommunityWrapperFactory` can also change the implementation contract.
- The owner of `SocietyVipManager` can define the Tiers, grant and revoke Community Tiers.
- The owner of the `CommunityWrapper` contract (which is not upgradeable) can manage (add and remove) badges for the community wrapper.
- In `SocietyProtocolBadges`, permissions are managed using OpenZeppelin's `AccessControl` contract. An account with the `CONTRACT_UPGRADER_ROLE` can upgrade `SocietyProtocolBadges`.
- A third system of permission control is regulated by Badges. For example, holders of a Creator badge for a community can deploy a new `CommunityWrapper` contract. Badges also (partly) regulate who can mint, transfer or burn other badges.
- Permissions regarding minting, transferring and burning of Badges are regulated by a mixed system consisting of roles (e.g. `COMMUNITY_MANAGER_ROLE` allows for some operations), the ownership of (other) Badges, custom permissions (such as `PERM_EVERYONE`), and hooks that can override all these permissions.

- Edit rights on Badges (setting the URI and the hook contract) are assigned directly to addresses via the `canEdit` mapping and are not transferable.

We believe this is problematic in two different ways:

First of all, using different systems to define permissions in the system (Ownable, AccessControl, the Badge system, direct assignments of editor rights, new permission types such as `PERM_EVERYONE`) leads to complexity, which makes it hard to understand which permissions apply where, and easy to make mistakes, both in the code (see for example SPB1) as well as future users will need to understand and manage these permissions.

For an example of the added complexity, consider a manager of a community that holds a Creator badge and has deployed a new `CommunityWrapper` contract. If the manager wants to retire and hand over management of the community to a different user, they will need to (a) transfer the Creator Badge, (b) transfer ownership of the wrapper contract to the new manager, but (c) they have no way of transferring the editor rights of the Creator badge (which allows them to set a hook contract, and so they keep full control over the Creator badge and therefore over the entire community).

A second observation regards separations of roles in the current setup. There are roles that are sensitive (e.g. those roles that allow accounts to upgrade a contract, which would typically be assigned to highly secured multisigs), while more typical admin roles which have specific and limited permissions (i.e. badge management for a specific community, which could in some cases be managed by an EOA account). It is good practice to define roles in such a way that such responsibilities are clearly separated. This is not the case in the current contracts: for example, in `SocietyVipManager`, the owner can both upgrade the contract, as well as manage the Tiers.

Recommendation: Simplify Role management. Define clear families of roles (for example, upgrader, role-admin, and operational roles).

Severity: Info

Resolution: The issue was not resolved

SocietyProtocolBadges.sol

SPB1. Profiles are not guaranteed to be “soul-bound” [high] [resolved]

By default, the owner of a “Profile” badge created with `createProfile` is an editor of that badge. That means that they can edit the hooks using `setBadgeHook`. Hooks have the highest priority when checking permissions in `_update`, and a user can set the hook in such a way that they can transfer, burn, and mint the token freely.

In other words, owners of a Profile badges can configure a hook contract that “allows everything”, and so, in particular, such badges are (a) not “soul-bound” (because the badges can be transferred to another account) and (b) the badges are not unique to an address (because any number can be minted to any address).

Recommendation: Depending on what your requirements are, you could either configure Profile tokens to not have an editor at all, or add additional restrictions on the use of Profile tokens that guarantee their uniqueness and non-transferability.

Severity: High

Resolution: Profile tokens have no editor set

SPB2. CreateProfile is vulnerable to reentrancy [medium] [resolved]

The implementation of `createProfile` allows for a re-entrancy attack. The crucial part of the code is this:

```
// Temporarily allow self-minting for the creation transaction
canMint[pid].push(PERM_SELF);
profileBadgeId[msg.sender] = pid;    // set before _mint (CEI)
_mint(msg.sender, pid, 1, "");
canMint[pid].pop();
```

The `_mint` function calls `checkOnERC1155Received` on `msg.sender`. If the sender is a malicious contract, it can implement a function that calls back to the `SocietyProtocolBadges` contract while the `canMint` mapping still includes the `PERM_SELF` permission. For example, the following code snippet would make it so that the calling contract receives 3 Profile badges instead of one:

```
function onERC1155Received(
    address,
    address,
    uint256,
    uint256,
    bytes calldata
) external override returns (bytes4) {
    if (!hasReentered) {
        hasReentered = true;
        address recipient = address(this);
        uint256 pid = badges.profileBadgeId(address(this));
        badges.mint(recipient, pid, 2, "");
    }
}
```

Within the context of the current contract, the impact of a contract owning more than one instance of a given profile token is small (probably the only place where the logic breaks is in `updateProfileURI`, which requires that the token is unique). However, third-party integrations (e.g. with Snapshot) may assume that profile tokens are unique.

Recommendation: Either refactor the code to strictly follow the Checks-Effects-Interactions pattern, or add a re-entrancy guard to the contract.

Severity: Medium

Resolution: There is now a re-entrancy guard that locks (further) profile minting during the createProfile call.

SPB3. Invitations do not implement EIP712 [medium][resolved]

The function `acceptInvite` accepts two different kind of signatures - either the inviter signs a EIP712 message of the form `abi.encode(INVITE_TYPEHASH, inviter, keccak256(bytes(message)))`, or the inviter just signs the `message`. The only requirement on the message is that the last 42 bytes must correspond to the address of the caller of the function.

There is nothing in `message` that binds the signature to the current contract or context. This means that `message` could be re-played as a valid invite if the contract is re-deployed, or deployed on another chain. There is also nothing in `message` that captures the intent of the signer - it is just a string with an address who could have signed (or be tricked to sign) a string with an address in a completely different context.

Also, the EIP712 message is replayable: the message does not contain the `EIP712Domain`, which fixes the signature to a contract address, chain and specific versions

Recommendation: Have inviters sign a complete EIP712 message, and only accept that as a valid invite. Also include the `EIP712Domain` info in the message. And if you have no specific reason to do otherwise, just accept a single format for the message (i.e. enforce that the message is exactly 42 bytes).

Severity: Medium

Resolution: The issue was resolved as recommended

SPB4. UpdateProfileURI allow for any badges with a supply of 1 to be updated by their owner [medium] [resolved]

The implementation of `updateProfileURI` has a name and a docstring that specifies that the function is meant to regulate that Profile owners can update the URI of their profile badges. However, the actual implementation allows for any badge that is unique to be updated by their owner.

```
/**
 * @notice Specifically for profile badges, allows the user holding it to
 * update their metadata link.
 */
```

```

function updateProfileURI(uint256 id, string memory newUri) external {
    // Allow update if sender owns the token and it's a unique NFT
(Profile)
    if (totalSupply(id) != 1 || balanceOf(msg.sender, id) != 1)
        revert NotProfileOwner();

    badges[id].metadataURI = newUri;
    emit URI(newUri, id);
}

```

Effectively, the current logic passes temporary control over the URI to whoever is the first to receive the badge. This may lead to consequences that are counter to expectations. For example, when an account is the first to receive a Badge, there is a window of time in which the Badge is unique, and during that time, the first owner of the badge can edit the URI. That URI may then be immutable as soon as the second instance of the Badge is created.

Recommendation: Use the `profileBadgeId` mapping to check if the badge to be updated is a Profile badge, instead of just requiring uniqueness of the badge.

Severity: Medium

Resolution: The issue was resolved as recommended

SPB5. Override of `balanceOf` by hook may break expectations about token ownership [low] [resolved]

A hook for a token may override `balanceOf`, and have it return a different amount than the amount of tokens actually held. Such an override breaks the link between an account actually holding a token (as represented internally in the `_balances` mapping, which regulates which account controls which tokens) and the result of calling `balanceOf`. It also breaks the link between the value of `totalSupply`, which can be different than the sum of all `balanceOf(holder, pid)` for each of the holders.

This may have negative consequences downstream. For example, some Snapshot strategies count the “weight” of a vote by comparing `balanceOf(holder, pid)` with the value of `totalSupply(pid)`

Recommendation: Do not override `balanceOf`, or, if you do, also allow for the hooks to override `totalSupply`, so that the two are consistent.

Severity: Low

Resolution: The developers state that the current behavior is as intended.

SPB6. Circular invite protection is only one-hop [low] [not resolved]

The invite code contains some logic to protect against “circular invites”:

```
if (inviter == msg.sender) revert SelfInvitation();  
if (invitedBy[inviter] == msg.sender) revert CircularInvitation();
```

This prevents direct $A \rightarrow B \rightarrow A$ cycles, but not larger ones. It is easy for a user to create a third sybil account and “auto-invite” themselves by passing the invite through this third account.

Recommendation: You could mint an “Invited” badge for each invitee, and require that the inviter has such a badge, but the invitee does not.

Severity: Low

Resolution: The issue was not resolved

SPB7. HookUpdated event is not emitted when hook is cleared [low] [resolved]

In `setBadgeHook`, the `HookUpdated` event is emitted unconditionally, but in `_createBadge`, the event is only emitted if `(hook != address(0))`. This inconsistency means indexers won't see a hook removal event when `_createBadge` is called with no hook, making off-chain tracking unreliable.

Recommendation: Emit `HookUpdated` every time the hook changes.

Severity: Low

Resolution: The issue was resolved as recommended

SPB8. `acceptInvite` does not check if the inviter is an existing protocol user [info] [resolved]

The `acceptInvite` function has a docstring that says that:

```
* @notice Accepts an invitation signed by an existing protocol user.
```

However, there is no check that the signer of the invite is a protocol user. Effectively, any user can invite any other user

Recommendation: Either update the docstring or the code.

Severity: Info

Resolution: The docstring was updated.

SPB9. Permissions may break common expectations regarding ERC1155 [info] [resolved]

If a badge uses the permission `PERM_EVERYONE` to manage transfer and burn permissions - then anyone can burn, or transfer, other users' tokens.

While this may be intended, we do want to point out that a common assumption for ERC20 and ERC1155 tokens is that "ownership" is defined by the fact that the owner is the only one that can move their tokens (in some cases, an admin can "freeze" tokens, but seeing tokens that can be arbitrarily moved is more rare) unless they explicitly allow other accounts or contracts to move their tokens.

A similar observation holds for badge-gated rules, and also for the hooks - these can all be configured in a way that anyone can burn, mint and transfer any token.

Another deviation from the ERC1155 (and ERC20) standard regarding permissions is that the approval logic is completely bypassed.

Recommendation:: While this is not something you need to necessarily change, but it should be clear to anyone that, say, uses `PERM_EVERYONE` for transfer (i.e. creates a token that can be transferred by anyone from any address to any address), does not create a token that can be meaningfully used for common use cases such as access, voting rights, proof of membership, etc.

Severity: Info

Resolution: The developers added checks that partially guard against misuse of these permissions, and in any case know about the issue and assert that the current behavior is as intended.

SPB10. The badgeCreated mapping can be safely removed [info] [resolved]

The `badgeCreated` mapping tracks which ids have been used. This mapping is not needed - whether a certain `badgeId` exists already can be easily checked by either consulting the `badges` mapping, or by checking if the `badgeId` is lower than the counter used for creating new ids:

```
function _badgeExists(uint256 id) internal view returns (bool) {
    return id > STARTING_BADGE_ID && id <= nextTokenId;
}
```

Recommendation: Remove the `badgeCreated` mapping

Severity: Info

Resolution: The issue was resolved as recommended

SPB11. Misleading variable names [info] [resolved]

The naming of the constant `STARTING_BADGE_ID` and state variable `nextTokenId` is slightly misleading: `STARTING_BADGE_ID` is *not* the id of the first badge (badge ids start with `STARTING_BADGE_ID+1`), and `nextTokenId` is *not* the id of the “next token”, but of the latest token that was created.

Recommendation: Align variable names with the actually implemented logic

Severity: Info

Resolution: The issue was not addressed directly, but comments are improved

SPB12. Editors, minters, transferers and burners can not be changed after badge creation [info] [resolved]

The mapping `canEdit`, `canMint`, `canTransfer` and `canBurn`, which regulate the token management, are, after a token has been created, immutable. This represents a security risk: if access to an account is lost or compromised, or people managing these accounts want to upgrade or transfer responsibility to other parties, these crucial rights cannot be transferred.

Recommendation: Implement a way to transfer these permissions.

Severity: Info

Resolution: The behavior remains unchanged - the team states that this is by design.

CommunityRegistry.sol

CR1. createCommunity is vulnerable to re-entry [low] [resolved]

The `createCommunity` function calls

```
badges.mint(msg.sender, creatorBadgeId, 1, "");  
badges.mint(msg.sender, memberBadgeId, 1, "");
```

Both these functions call `onERC1155Received` on `msg.sender`, if that account is a contract.

The “Checks-Effects-Interactions” pattern requires that all state changes are done before any external calls, to minimize the risk of re-entry into the contract while it is in an inconsistent or unexpected state.

Because implementation creates the community after these calls, the function does not follow this pattern.

Recommendation: Although we do not see an attack vector here, we do recommend that you follow the CEI pattern and create the community before doing any external calls.

Severity: Low

Resolution: The issue was resolved as recommended

CR2. Permissions as described in docstring do not correspond with actual permissions [info] [resolved]

```
* Creator badge permissions:
* - canMint: [] - only registry (via COMMUNITY_MANAGER_ROLE) can mint.
* - canTransfer: [PERM_SELF] - only the current holder can transfer.
* - canBurn: [] - non-burnable.
*
* Member badge permissions:
* - canMint: [creatorBadgeId] - only Creator badge holder can mint.
* - canTransfer: [] - soulbound.
* - canBurn: [creatorBadgeId] - Creator badge holder can burn.
```

This comment is misleading because:

(a) not only the registry, but any account that has the `COMMUNITY_MANAGER_ROLE` can mint `Creator` badges. In addition, the comment suggests that the registry can mint new badges, but it does not have any implemented functionality to do that

(b) The comment states correctly that the Creator Badge is not “burnable”, but it should be clear that the owner of the token can transfer the token to an address such as `0x01`, which is functionally the same as burning the token

(c) The comment says that the Member badges are “soul-bound”, as they cannot be transferred. But for fungible tokens, burning from one address and subsequently minting to another address is functionally the same as transferring the token.

Another observation is that Creator badges, Member badges, and Community Badges, are created with the registry contract itself as an editor. These edit rights are not exercisable, because the registry has no functionality to edit these badges. The registry contract is upgradeable, however, so these rights may be exercised in a future version of the contract, but currently, they are moot.

Recommendation: Update the documentation, and make sure the code matches your intentions.

Severity: Info

Resolution: The issue was resolved as recommended

CommunityWrapper.sol

CW1. Implementation of `totalSupply` limits possible Snapshot strategies that could be used [info] [resolved]

In the current implementation of the `CommunityWrapper`, `totalSupply` always returns 0. This limits which Snapshot strategies can be used - examples would be supply-weighted strategies, strategies that implement a quorum based on `totalSupply`. This limitation seems unnecessary - because the Badges already inherit from `ERC1155Supply`, the `totalSupply` function is available for each of the allowed Badges.

Recommendation: redefine `totalSupply` to return the sum of the total supply for each of the badges (But also see SPB5)

Severity: Info

Resolution: The developers stated that they are aware of the issue, and that this is by design.

CW2. Approval does not throw an error [info] [resolved]

In the implementation, a call `transfer` and `transferFrom` will revert with a `TransfersDisabled` errors. But the `approve` function will still work as implemented in the `ERC20` implementation from OpenZeppelin.

Recommendation: For consistent behavior, also throw an error when `approve` is called.

Severity: Info

Resolution: The issue was resolved as recommended

CommunityWrapperFactory.sol

CW1. Sanity check on implementation contract can be improved [low] [resolved]

Currently, the functions `initialize` and `setWrapperImplementation` check that the `implementation` provided is not a 0 address:

```
require(_wrapperImplementation != address(0), "Invalid implementation");
```

This check is quite weak, and specifically, it allows for an EOA or other contract to be deployed as the implementation contract.

Recommendation: Require that there is actually deployed code at the given address:

```
require(_wrapperImplementation.code.length > 0, "Invalid implementation");
```

Or, to be even more strict, you could implement an ERC165 interface for the wrapper contract, and require that the implementation implements that interface.

Severity: Low

Resolution: The issue was resolved as recommended

SocietyVipManager.sol

SVM1. Staked Tokens can get locked forever [high] [resolved]

If a user has tokens in a `userlock` that is expired, and then calls `lock` with a new amount, the expired lock is overwritten, and the tokens in the expired lock are stuck in the contract permanently.

The issue is in the following code snippet:

```
if (userLock.amount > 0 && block.timestamp < userLock.unlockTime) {
    userLock.amount += amount;
    uint256 newUnlockTime = block.timestamp + duration;
    if (newUnlockTime > userLock.unlockTime) {
        userLock.unlockTime = newUnlockTime;
    }
} else {
    // New lock or expired lock – start fresh
    userLock.amount = amount;
    userLock.unlockTime = block.timestamp + duration;
}
```

Recommendation: Either treat expired locks in the same way as non-expired ones (i.e. remove the check on the `unlockTime` from the “if” clause), or, alternatively, throw an error and ask the user to unlock first.

Severity: High

Resolution: The issue was resolved as recommended